

1. 비동기 방식

현대 웹 애플리케이션에서는 비동기, RESTful 등의 개념을 중요하게 생각합니다. 이들에 대해서 살펴보겠습니다.

웹 개발에서 비동기(Asynchronous) 방식은 요청을 보낸 후 응답을 기다리지 않고 다른 작업을 계속 수행할 수 있도록 하는 방식입니다. 비동기의 반대는 동기(Synchronous) 방식으로, 다음은 차이점입니다.

비동기(Asynchronous)	동기(Synchronous)
클라이언트는 서버로 요청을 보낸 후 응답을 기다리지 않고 다음 작업을 진행합니다.	클라이언트는 서버로 요청을 보낸 후 응답이 올 때까지 기다린 후 다음 작업을 진행합니다.

비동기와 동기 방식을 보다 쉽게 이해하기 위해서 일상의 예를 들면 비동기 방식은 음식을 주문하고 음식이 조리되는 동안 다른 일(예. 친구와 대화, SNS 활동, 손 씻기 등)을 진행합니다. 반면 동기 방식은 음식을 주문하고 조리가 끝날 때까지 다른 일을 하지 않고 기다립니다. 현대 웹 애플리케이션에서 비동기 방식이 중요한 이유는 빠른 응답성과 효율적인 자원 활용 때문입니다.

▶ 비동기 방식의 장점

1) 서버의 부하 감소: 클라이언트는 요청을 보내고, 서버의 응답을 기다리지 않고 다른 작업을 계속할 수 있습니다. 예를 들어, 식당 키오스크는 고객의 주문을 받고 음식이 조리되는 동안 다음 고객의 주문을 받을 수 있어 고객들이 길게 줄 서서 기다릴 필요가 없게 됩니다. 만약 동기 방식이라면 음식이 만들어지는 동안 추가 주문을 받을 수 없어 고객들이 길게 줄 서게 됩니다.

2) 빠른 사용자 경험: 클라이언트 요청 시 화면이 멈추지 않고 부드럽게 동작할 수 있습니다. 동기 방식의 경우 사용자가 버튼을 클릭해서 서버에 요청하면, 서버가 응답할 때까지 웹페이지가 멈춘 상태로 대기하기 때문에 사용자는 아무것도 할 수 없게 됩니다. 서버 응답이 느리면 웹사이트가 정지된 것처럼 보일 수 있고, 사용자는 불편하고 답답함으로 스트레스를 받게 됩니다. 이런 스트레스를 해소하기 위해서 비동기 방식을 사용하면 사용자가 버튼을 클릭해서 서버에 요청한 후 서버 응답을 기다리지 않고 웹사이트는 계속 동작합니다. 이후 서버가 응답하면, 응답에 대한 화면 일부만을 업데이트합니다. 결국 웹페이지가 부드럽게 작동하며, 빠른 느낌을 주게됩니다.

3) 병렬 처리 가능: 여러 개의 요청을 동시에 처리 가능합니다. 클라이언트는 서버의 응답을 기다리지 않기 때문에 여러 개의 요청을 연속해서 보낼 수 있습니다. 동기 방식이라면 하나의 요청에 대한 응답이 완료된 후 다음 요청을 해야하는 불편함이 있습니다.

비동기 방식은 사용자 경험(UX)을 향상시키고, 서버 부하를 줄이며, 동시에 여러 요청을 처리할 수 있어 현대 웹 애플리케이션에서 더 많이 사용됩니다. 우리가 만들고 있는 캘린더 프로

젝트에서 ‘관리자 권한 변경’ 기능도 비동기 방식으로 구현합니다.

2. REST API

비동기 방식이 무엇인지 알았는데요, 비동기 방식을 구현하는 방법은 다양하게 있습니다.

2-1. REST API (AJAX, Fetch, Axios 등)

가장 많이 사용되는 방법으로 클라이언트(브라우저)가 AJAX, Fetch, Axios 같은 기술 또는 라이브러리를 이용해서 비동기 요청을 보내면 서버가 처리한 후 결과를 비동기적으로 반환합니다.

▶ AJAX (Asynchronous JavaScript and XML)

AJAX는 특정한 라이브러리나 모듈이 아니라, 비동기적으로 데이터를 주고받는 기술을 의미합니다. 예전에는 XMLHttpRequest 객체를 사용해 AJAX 요청을 보냈지만, 요즘은 Fetch나 Axios로 대체되었습니다.

샘플 코드

```
var xhr = new XMLHttpRequest();
xhr.open("GET", "https://api.example.com/data", true);
xhr.onreadystatechange = function() {
    if (xhr.readyState === 4 && xhr.status === 200) {
        console.log(xhr.responseText);
    }
};
xhr.send();
```

XMLHttpRequest를 이용하는 방법은 코드가 길고, 콜백 함수가 많아지면 구조가 복잡해지는 단점으로 인해 최근에는 거의 사용하지 않습니다. XMLHttpRequest의 단점을 보완해서 jQuery의 \$.ajax()가 만들어져서 인기를 많이 얻었지만 이 또한 최근에는 많이 사용하지 않습니다. \$.ajax()는 XMLHttpRequest에 비해서 가독성이 좋습니다.

샘플 코드

```
$.ajax({
    url: "https://api.example.com/data",
    method: "GET",
    success: function(data) {
        console.log(data);
    },
    error: function(error) {
        console.error(error);
    }
});
```

▶ Fetch API

JavaScript 기본 내장 기능으로 `fetch()` 함수를 사용해서 요청을 보낼 수 있습니다. `$.ajax()` 는 jQuery 라이브러리를 별도로 설치해야 하지만 Fetch API는 별도 설치 없이 바로 사용 가능하다는 점과 Promise 기반으로 동작하는 장점이 있습니다. Promise를 사용하면 비동기 코드를 더 깔끔하고 읽기 쉽게 작성할 수 있고, `then()`과 `catch()`로 비동기 응답 처리를 간단하게 할 수 있어 코드 흐름이 명확해집니다.

샘플 코드

```
fetch('https://api.example.com/data')
  .then(response => response.json())           // 응답을 JSON으로 변환
  .then(data => console.log(data))             // 데이터 처리
  .catch(error => console.error(error));       // 오류 처리
```

▶ Axios

비동기 요청을 도와주는 외부 라이브러리로 별도로 설치(`npm install axios`)해야 하기 때문에 일반적으로 모듈이라고 합니다.

샘플 코드

```
import axios from 'axios'; // Axios 모듈 импорт(Fetch API와 차이점)

axios.get('https://api.example.com/data')
  .then(response => {
    console.log(response.data); // 응답 데이터 처리
  })
  .catch(error => {
    console.error(error);      // 오류 처리
  });
```

Fetch에서는 `.json()` 메서드를 사용해서 응답 데이터를 JSON으로 변환했지만, Axios는 `response.data` 속성을 통해 직접 데이터를 가져올 수 있습니다. 다음은 Fetch와 Axios의 차이점과 공통점입니다.

	Axios	Fetch API
응답 데이터	<code>response.data</code>	<code>response.json()</code>
HTTP 상태 코드	자동 처리 -200번대: <code>then()</code> -400번대/500번대: <code>catch()</code>	직접 처리 - <code>response.ok</code> 사용
응답 처리	Promise 반환, <code>then()</code> 사용	
에러 처리	<code>catch()</code> 로 처리	

일부 개발사에서는 Fetch API가 Axios보다 성능 면에서 뛰어나다고 선호하기도 하지만 실제로 성능 면에서 더 좋은지에 대한 논란은 계속되고 있습니다. 오히려 성능보다는 편의성과 구현 방식에 있어서 Fetch API가 Axios보다 미미하게 좋지만 이 또한 개인적인 견해일 수 있습니다. 따라서 Fetch API가 Axios 중에서 어떤 방법을 선택해야 하는지는 개인 또는 조직의 몫일 수 있습니다.

2-2. WebSocket (실시간 양방향 통신)

REST API는 요청이 있을 때만 응답하지만, WebSocket은 서버와 클라이언트가 항상 연결된 상태로 데이터를 주고받을 수 있습니다. 예로 실시간 채팅, 스포츠 경기 중계 등이 있습니다.

2-3. Server-Sent Events (SSE)

서버가 클라이언트에 일방적으로 데이터를 푸시하는 방식으로, 뉴스 속보, 실시간 주식 정보, 타임 세일 알림 등이 있습니다.

본 강의에서는 비동기 방식을 구현하는 다양한 방법 중 가장 많이 사용하고, 웹 개발자가 반드시 알고 있어야 하는 REST API에 집중하겠습니다.

3. RESTful

REST(Representational State Transfer)는 웹에서 데이터를 주고받는 아키텍처 스타일 뜻합니다. 그리고 REST 원칙을 잘 지킨 API를 RESTful이라고 합니다. 말이 어렵죠~ 쉽게 설명해 보겠습니다.

RESTful은 특정 기능을 구현하는 ‘방법’이 아니고, ‘자원(데이터)에 집중하자!’라는 ‘개념적인 원칙’입니다. 예를 들어 보겠습니다. 클라이언트에서 서버로 id가 1인 사용자 조회를 요청할 때 요청 URL을 ‘/getUser?id=1’로 할 수 있습니다. 이것이 잘못된 것은 아니지만, 요청 URL을 자세히 분석해 보면 완벽하게 자원(데이터)에 집중하지 못하고 있습니다. 이유는 ‘getUser’는 우리말로 ‘User를 가져와라!’로 ‘자원’ 중심이 아니라 ‘동작’ 중심입니다. 이렇게 동작에 중심인 것을 ‘일반적인 HTTP API’라고 하는데요, RESTful은 자원 중심이기 때문에 요청 URL을 ‘/users/1’와 같이 설계해야 맞습니다. 또한 ‘get’과 같은 표현은 ‘HTTP 메서드’를 이용하는데요, HTTP 메서드에는 ‘GET, POST, PUT, DELETE’ 등이 있습니다.

	예제	설명
RESTful	GET /users/1	리소스 중심: 1번 사용자 정보를 가져와라!
일반적인 HTTP API	GET /getUser?id=1	동작 중심: getUser라는 기능을 실행해라!

지금까지의 내용을 정리하면 RESTful은 웹 주소(URL)와 HTTP 메서드를 이용해 데이터를 주고받는 규칙입니다. 이렇게 하면 서버랑 클라이언트가 보다 간결하게 소통할 수 있습니다. 입문자의 경우 아직 RESTful의 개념이 정확하게 와닿지 않을 수 있습니다. 다음 샘플코드로 이해합니다.

샘플 코드

```
@Controller
@RequestMapping("/users")
public class UserController {

    @GetMapping
    @ResponseBody
    public String getUsers() {
        return "회원 목록";
    }

    @PostMapping
    @ResponseBody
    public String addUser() {
        return "회원 추가";
    }

    @PutMapping("/{id}")
    @ResponseBody
    public String updateUser(@PathVariable int id) {
        return id + "번 회원 수정";
    }

    @DeleteMapping("/{id}")
    @ResponseBody
    public String deleteUser(@PathVariable int id) {
        return id + "번 회원 삭제";
    }
}
```

UserController의 메서드들은 공통으로 '/users' 요청을 처리합니다. 차이점은 getUsers() HTTP GET 방식으로 회원 목록을 조회(get)하고, addUser()는 HTTP POST 방식으로 회원을 추가(post)합니다. updateUser()와 deleteUser()는 HTTP PUT과 DELETE 방식으로 회원 정보를 수정하거나 삭제합니다.

개발자는 RESTful 원칙을 따르는 URL을 설계할 때 많은 고민을 하곤 하는데요, 중요한 것은 자원(Resource) 중심으로 URL을 표현하는 것입니다. ID가 'abc'인 회원의 지난달 판매내역을 조회하는 RESTful한 URL을 생각해 보겠습니다.

코드
GET /getSalesHistory?userId=abc&month=2025-01

위 URL이 틀린 것은 아니지만 RESTful(자원에 집중한) URL은 아닙니다. 이유는 getSalesHistory처럼 동작이 URL에 포함되면 RESTful하지 않습니다. 동작은 HTTP 메시드로 구분해야 합니다.

다음은 RESTful(자원에 집중한) URL로 개선한 코드입니다.

코드
GET /users/abc/sales?month=2025-01

‘/users/abc/sales’는 ‘abc’ 회원의 판매 내역이라는 자원을 나타내고 있습니다. 그리고 ‘?month=2025-01’는 특정 기간을 필터링하는 쿼리 파라미터입니다. 위 요청 URL은 올바른 RESTful URL 설계라고 볼 수 있습니다.

방금 위 코드를 다음과 같이 경로 변수만을 이용해서 설계할 수도 있습니다.

코드
GET /users/abc/sales/2025/1

쿼리 파라미터와 경로 변수 중 어떤 게 더 RESTful할까요? RESTful URL 설계에서 중요한 기준은 ‘이 값이 자원의 계층적 구조를 나타내는가?’를 판단하는 것입니다. 데이터베이스에서 연도와 월이 계층적 구조를 나타낸다면 경로 변수(‘/2025/1’)가 좋고, 특정 자원을 필터링 해야한다면 쿼리파라미터가(month=2025-01) 적합하다고 볼 수 있습니다.

	설계	내용
쿼리 파라미터	/users/abc/sales?month=2025-01	특정 자원(판매 내역)에서 필터링할 때 적합
경로 변수	/users/abc/sales/2025/1	연도와 월이 계층적 구조를 나타낼 때 적합

/users/abc/sales?month=2025-01: 판매 내역에서 2024년 1월만 필터링한다.

/users/abc/sales/2025/1: 사용자 abc의 2025년 1월 판매 내역을 조회한다.(계층적 구조(사용자 > 연도 > 월))

지금까지 비동기 방식과 RESTful에 대해서 살펴보았는데, 비동기 방식은 쉽게 이해되지만

RESTful은 다소 알쏭달쏭할 수 있습니다. 앞에서 말씀드렸듯이 RESTful은 기술이 아닌 '자원에 집중하자!'라는 개념으로 한 번에 이해하기란 어렵습니다. 또한 실무에서도 비동기, REST API, RESTful을 동일한 용어로 사용하기도 합니다. 따라서 여러분은 '현대 웹 애플리케이션에서 비동기가 인기를 얻고 있으며, 이를 구현하기 위해서 REST API 방법을 주로 사용한다. 그리고 요청 URL을 설계할 때 최대한 자원에 집중(RESTful) 하자!'라고 이해하면 됩니다.

4. 비동기 구현 방법

Spring Boot에서는 비동기 방식의 데이터 요청을 위해 fetch API와 @ResponseBody를 사용합니다.

▶ 클라이언트: fetch API 사용

fetch API는 자바스크립트에서 제공하는 비동기 요청 함수입니다.

샘플 코드	
fetch('/api/message')	// 서버에 요청
.then(response => response.json())	// 응답을 JSON으로 변환
.then(data => console.log(data))	// 응답 데이터 활용
.catch(error => console.error('Error:', error));	

▶ 서버: @ResponseBody 사용

Spring Boot에서는 @ResponseBody를 사용해서 JSON 데이터를 반환합니다.

샘플 코드	
@Controller	
@RequestMapping("/api")	
public class MessageController {	
@GetMapping("/message")	// '/api/message' GET 요청을 처리
@ResponseBody	// JSON 데이터 반환
public String getMessage() {	
return "응답 데이터";	
}	
}	

비동기 방식과 RESTful에 대한 개념은 다소 어렵지만 실제 비동기 구현 방법은 간단합니다.